

基于 Verilog 的 I2C 接口组件库设计与分析

摘要

随着 FPGA 在嵌入式控制、数据采集、工业通信以及片上系统原型验证中的应用不断扩大，外设接口的可复用设计成为数字系统开发中的重要问题。I2C 总线由于引脚数量少、协议结构清晰、外围器件支持广泛，常用于连接 EEPROM、时钟芯片、传感器、电源管理芯片和低速控制外设。在 FPGA 系统中，如果每个项目都重新实现 I2C 通信逻辑，不仅会增加开发周期，也容易在时序控制、应答检测、总线仲裁、时钟拉伸和异常处理等方面引入重复错误。因此，设计一套结构清晰、可复用、便于片上总线集成并配套验证环境的 I2C 接口组件库，具有较强的工程意义和教学参考价值。

本文以 `verilog-i2c-master` 项目为研究对象，围绕其 Verilog I2C 接口组件库的功能结构、模块设计、关键实现逻辑和验证体系展开分析。该项目提供了 I2C 主控制器、I2C 从控制器、AXI-Lite 主机包装层、Wishbone 主机包装层、I2C 到片上总线桥接模块、上电初始化模块、单寄存器从设备示例以及配套的 MyHDL 协同仿真测试平台。本文首先介绍 I2C 总线在 FPGA 外设控制中的应用背景，分析项目面向的功能需求与非功能约束；随后从总体架构角度说明项目采用“核心协议引擎、片上总线包装层、应用示例模块、测试支撑层”的分层组织方式；接着重点分析 `i2c_master.v`、`i2c_slave.v`、`i2c_master_axil.v`、`i2c_slave_axil_master.v`、`i2c_init.v` 等核心模块的功能、接口、数据流和设计思想；最后结合项目测试平台，讨论其验证方式、已有材料能够支持的结果以及后续改进方向。

研究表明，该项目具有较好的模块化、参数化和复用性。其核心模块通过 AXI Stream 风格接口实现控制逻辑与协议时序解耦，并通过 AXI-Lite、Wishbone 等包装层适配常见片上总线系统；测试平台则利用 Python/MyHDL 总线模型与 Icarus Verilog 协同仿真，对主机、从机、初始化流程和桥接场景进行验证设计。与此同时，项目材料也显示该仓库已经停止维护，且缺少综合报告、时序报告、板级验证结果和资源利用率数据。因此，本文将将其定位为 FPGA I2C 接口设计的学习参考、二次移植基础和模块化 HDL 设计案例，而不将其直接视为未经进一步验证即可用于生产环境的最终方案。

关键词：I2C 总线，Verilog，FPGA，AXI-Lite，Wishbone，协同仿真

Abstract

With the increasing use of FPGA devices in embedded control, data acquisition, industrial communication, and system-on-chip prototyping, reusable peripheral interface modules have become important in digital system development. The I2C bus is widely used for connecting EEPROMs, clock devices, sensors, power-management chips, and low-speed control peripherals because it requires few pins and has a relatively simple protocol structure. In FPGA systems, repeatedly developing I2C communication logic for each project may increase development time and introduce errors in timing control, acknowledge detection, clock stretching, and exception handling. Therefore, it is meaningful to study a reusable I2C interface component library that is modular, parameterized, easy to integrate with on-chip buses, and supported by a verification environment.

This thesis studies the `verilog-i2c-master` project and analyzes its Verilog-based I2C interface component library in terms of functional requirements, system architecture, module design, implementation logic, and verification methods. The project provides an I2C master controller, an I2C slave controller, AXI-Lite and Wishbone wrappers, I2C-to-on-chip-bus bridge modules, an initialization state-machine template, a single-register I2C slave example, and a MyHDL-based co-simulation testbench. This thesis first introduces the application background of I2C communication in FPGA peripheral control, and then analyzes the project requirements and design constraints. It further explains the layered architecture of the project, including the core protocol engines, bus wrappers, application modules, and verification support. The core modules such as `i2c_master.v`, `i2c_slave.v`, `i2c_master_axil.v`, `i2c_slave_axil_master.v`, and `i2c_init.v` are discussed in detail. Finally, the verification design and limitations of the project are summarized.

The analysis shows that the project has good modularity, parameterization, and reusability. The core modules use AXI Stream style interfaces to decouple protocol timing from control logic, while the AXI-Lite and Wishbone wrappers make the design suitable for common FPGA-based systems. The verification environment combines Python/MyHDL bus models with Icarus Verilog co-simulation. However, the available materials also indicate that the repository has been deprecated and lacks synthesis reports, timing reports, board-level validation results, and resource utilization data. Therefore, this thesis regards the project as a useful learning reference and reusable design basis rather than a production-ready solution without further validation.

Key words: I2C bus, Verilog, FPGA, AXI-Lite, Wishbone, co-simulation

1 绪论

1.1 研究背景

在现代嵌入式系统和数字逻辑系统中，FPGA 既可以承担高速数据处理、协议转换、实时控制等任务，也经常需要与板级外设进行低速控制通信。相较于高速串行链路或复杂并行总线，I2C 总线具有线路少、连接简单、支持多从设备、外围芯片普遍兼容等特点，因而被广泛用于芯片配置、传感器读取、时钟芯片控制、电源管理和寄存器访问等场景。对于 FPGA 设计而言，I2C 通信逻辑通常不是系统的主功能模块，却是系统启动、外设配置和运行状态管理不可缺少的基础组成部分。

I2C 总线虽然概念上较为简单，但在 HDL 实现中仍涉及较多细节。主控制器需要产生起始条件、停止条件、地址阶段、读写阶段和 ACK/NACK 检测；从控制器需要完成地址匹配、应答响应、数据接收、数据发送和时钟拉伸；当系统需要通过片上总线控制 I2C 外设时，还需要在协议控制器外部增加寄存器映射、FIFO 缓冲、命令接口和状态反馈机制。如果缺乏成熟组件，开发者往往需要在不同项目中重复编写类似逻辑，这不仅降低开发效率，也使验证工作分散且不完整。

`verilog-i2c-master` 项目正是面向这一类需求的 Verilog I2C 接口组件库。根据项目报告，该项目提供了一组可复用的 I2C 主从控制器、片上总线包装层、初始化状态机和测试平台，目标是在 Verilog 设计中快速集成 I2C 通信能力。该项目既包含核心协议模块，也包含 AXI-Lite 和 Wishbone 适配层，还配套了 Python/MyHDL 行为模型与 Icarus Verilog 协同仿真环境。这种组织方式使其不仅可作为工程复用模块，也适合作为本科阶段学习 HDL 模块化设计、总线接口封装和数字系统验证方法的案例。

1.2 研究目的与意义

本文的研究目的在于对 `verilog-i2c-master` 项目进行系统化论文整理，从本科毕业论文的角度说明该项目的设计背景、需求分析、总体架构、模块划分、实现逻辑和验证方式。本文并非重新从零实现一个 I2C 控制器，而是在项目报告所总结的事实基础上，将项目内容转化为符合毕业论文表达习惯的技术文档，使项目的工程价值、设计思路和不足之处得到较完整呈现。

从工程角度看，本文的意义主要体现在三个方面。第一，I2C 控制器是 FPGA 系统中常见而基础的接口组件，通过分析该项目能够帮助理解协议控制器如何从位级时序扩展为可复用的片上总线外设。第二，该项目同时支持 AXI-Lite 和 Wishbone 等总线封装，体现了接口适配和模块复用的设计思想，具有较好的工程教学价值。第三，项目包含 MyHDL 协同仿真测试平台，展示了 HDL 模块验证不应停留在语法编译层面，而应围绕协议行为、数据流和边界情况构造测试激励。

从学习角度看，该项目适合用于理解 FPGA 外设接口组件库的构建方法。一个可复用 HDL 项目通常不应只包含单一 RTL 文件，而应包含清晰的接口定义、参数化配置、上层包装、示例应用和测试环境。`verilog-i2c-master` 的目录结构和模块划分较好地体现了这一点。通过论文方式展开分析，可以帮助读者形成从“协议功能”到“系统集成”再到“验证支撑”的完整认识。

1.3 国内外研究与应用概况

I2C 总线由飞利浦公司提出，长期用于低速片间通信。由于其只需要串行数据线 SDA 和串行时钟线 SCL 两条信号线，且支持多个从设备通过地址区分，因此在嵌入式硬件平台中十分常见。在 MCU 应用中，I2C 控制器往往由芯片内部外设提供，开发者通过寄存器或驱动库完成访问；而在 FPGA 应用中，I2C 控制器可能需要根据系统需求自行实现或集成第三方 HDL IP。

在 FPGA 设计实践中，I2C 控制器可分为简单专用型和通用可复用型。简单专用型控制器通常只面向某一颗外设，在状态机中固定写入初始化序列，适合规模较小、需求固定的设计。通用可复用型控制器则提供命令接口、数据接口、状态反馈和参数化配置，可以由上层 CPU、软核处理器或其他逻辑模块动态控制。`verilog-i2c-master` 更接近后者，它不仅实现基础主从协议，还将控制能力封装到 AXI-Lite 和 Wishbone 等片上总线环境中，从而提高项目复用价值。

在验证方法方面，传统 HDL 项目常使用 Verilog testbench 进行仿真，但当协议行为较复杂时，纯 Verilog 激励编写成本较高。Python/MyHDL、cocotb、SystemVerilog/UVM 等方法为 HDL 验证提供了更灵活的测试组织方式。项目报告显示，`verilog-i2c-master` 使用 MyHDL 行为模型和 Icarus Verilog 协同仿真，通过 Python 负责测试激励和总线模型，Verilog 负责被测模块实现。这种方式便于构造较复杂的总线交互场景，也便于在测试脚本中加入断言和数据检查。

1.4 研究内容

本文围绕 `verilog-i2c-master` 项目展开，主要研究内容如下。

第一，分析项目背景与需求。本文根据项目报告总结该项目的应用场景，说明其面向 FPGA 外设控制、上电初始化、片上总线与外部微控制器桥接、寄存器访问和数据交互等需求。

第二，分析系统总体架构。项目采用“核心 I2C 协议引擎 + 片上总线包装层 + 应用层示例 + 测试平台”的结构。本文将从模块层次、目录结构和数据流角度说明该架构如何支持复用和扩展。

第三，分析核心模块设计。本文重点讨论 I2C 主控制器、I2C 从控制器、AXI-Lite 主机包装层、Wishbone 主机包装层、I2C 从设备桥接层、初始化模块、单寄存器从设备和 AXI Stream FIFO 等模块的功能定位与接口特点。

第四，分析实现与验证。本文根据项目报告说明其关键实现逻辑，包括协议状态机、流式接口解耦、寄存器映射、总线桥接、初始化脚本化执行和协同仿真测试方式。同时，本文谨慎区分“测试平台存在”和“实际测试结果已运行通过”两类结论，避免把未实际执行的仿真结果写成事实。

第五，总结项目不足和改进方向。项目报告明确指出该仓库已停止维护，且缺少综合结果、时序收敛报告、资源利用率报告和特定 FPGA 板卡工程。因此，本文将在最后提出后续改进建议。

1.5 论文组织结构

本文共分为七章。第一章为绪论，介绍研究背景、目的意义、应用概况、研究内容和论文结构。第二章介绍相关技术基础，包括 I2C 总线、Verilog HDL、FPGA 模块化设计、AXI-Lite、Wishbone、AXI Stream 和协同仿真方法。第三章进行系统需求分析，说明项目的功能需求、非功能需求和设计约束。第四章介绍系统总体设计，包括工程结构、模块层次、接口关系和数据流程。第五章进行详细设计与实现分析，重点说明各核心模块的功能和关键逻辑。第六章介绍测试与验证设计，分析测试平台组成、验证方法、已有材料能够支持的结果和不足。第七章为总结与展望，对全文进行归纳并提出后续优化方向。

2 相关技术与理论基础

2.1 I2C 总线协议概述

I2C 总线是一种同步串行通信总线，通常由串行时钟线 SCL 和串行数据线 SDA 构成。总线上的设备通过开漏方式连接，依靠上拉电阻维持高电平，任一设备均可主动拉低信号线。该特性使多个设备共享同一总线成为可能，同时也要求控制器在输出高电平时并不直接驱动高电平，而是释放总线，由上拉电阻或外部电路使总线回到高电平。

I2C 通信通常由主设备发起。主设备产生 SCL 时钟，并通过 SDA 发送起始条件、从设备地址、读写位、数据字节和停止条件。从设备根据地址判断是否响应，并在每个字节之后返回 ACK 或 NACK。对于读操作，从设备在主设备提供的时钟下发送数据；对于写操作，主设备发送数据，从设备接收并应答。

在 HDL 实现中，I2C 的关键在于正确生成和检测时序。起始条件通常表现为 SCL 为高时 SDA 从高变低，停止条件表现为 SCL 为高时 SDA 从低变高。数据位一般在 SCL 低电平期间变化，在 SCL 高电平期间保持稳定。控制器还需要处理 ACK 检测、重复起始、连续读写、时钟拉伸和总线忙状态等情况。项目报告中指出，`i2c_master.v` 支持读、写、连续写、起始/重复起始、停止等操作，并通过 `missed_ack` 状态输出检测从机未应答；`i2c_slave.v` 支持 I2C 读写事务解析，并能在读取数据尚未准备好时通过保持 SCL 为低实现时钟拉伸。这些功能均与 I2C 协议的典型要求一致。

2.2 Verilog HDL 与 FPGA 设计

Verilog HDL 是数字逻辑设计中常用的硬件描述语言，可用于描述组合逻辑、时序逻辑、状态机、模块接口以及层次化电路结构。FPGA 设计通常以 Verilog 或 VHDL 为主要输入，通过综合工具将 RTL 描述映射到目标器件的查找表、触发器、块 RAM、DSP 和 I/O 资源上。

与软件程序不同，HDL 描述的是硬件并行结构。因此，在设计接口控制器时，需要关注时钟域、复位逻辑、状态跳转、信号握手、总线时序和资源消耗等问题。I2C 控制器虽然速率相对较低，但仍需要保证状态机在不同输入条件下稳定运行。项目使用 Verilog 2001 编写，有利于在常见 FPGA 工具链中复用。项目报告还指出，多个模块支持参数化配置，例如数据宽度、地址宽度、FIFO 深度、输入滤波长度和分频参数等，这体现了 HDL 组件库设计中常见的可配置思想。

2.3 模块化与参数化设计

模块化设计是 HDL 工程复用的基础。一个复杂接口组件可以拆分为协议核心、数据缓冲、寄存器接口、总线桥接和测试支撑等多个模块。通过清晰的接口边界，不同模块可以独立开发、仿真和复用。参数化设计则允许同一模块在不同场景下通过参数调整位宽、深度、地址范围或功能开关，减少重复代码。

`verilog-i2c-master` 的模块划分体现了这种思想。`i2c_master.v` 和 `i2c_slave.v` 是协议核心；`i2c_master_axil.v`、`i2c_master_wbs_8.v` 和 `i2c_master_wbs_16.v` 是主机包装层；`i2c_slave_axil_master.v` 和 `i2c_slave_wbm.v` 是从机桥接层；`i2c_init.v` 和 `i2c_single_reg.v` 面向具体应用场景；`axis_fifo.v` 则作为公共缓冲模块服务于数据流解耦。这样的划分使项目既能在简单设计中直接使用核心模块，也能在片上总线系统中通过包装层进行集成。

2.4 AXI Stream、AXI-Lite 与 Wishbone 总线

AXI Stream 是一种面向流式数据传输的接口风格，常用于模块之间传递连续数据或命令。其核心思想是通过 `valid` 和 `ready` 信号进行握手，发送方在数据有效时拉高 `valid`，接收方在可以接收时拉高 `ready`，当二者同时有效时完成一次传输。在 I2C 控制器中，AXI Stream 风格接口可以用于输入命令、写入数据和输出读回数据，从而使上层逻辑不必直接关心 I2C 位级时序。

AXI-Lite 是 AXI 总线族中的轻量级寄存器访问接口，适合低带宽控制寄存器读写。FPGA 或 SoC 系统中，软核 CPU 或片上控制器可通过 AXI-Lite 访问外设寄存器。项目中的 `i2c_master_axil.v` 将 I2C 主控制器包装为 AXI-Lite 从设备，提供状态、命令、数据和分频等寄存器，使 CPU 可以用寄存器读写方式控制 I2C 通信。

Wishbone 是另一种常见开放片上总线规范，在许多开源软核和 FPGA 项目中使用。项目提供 `i2c_master_wbs_8.v` 和 `i2c_master_wbs_16.v` 两种 Wishbone 主机包装层，分别适配不同数据位宽的系统。项目还提供 `i2c_slave_wbm.v`，使外部 I2C 主设备能够间接访问 FPGA 内部 Wishbone 总线。这说明项目不仅关注单一总线环境，而是希望在多种开源硬件系统中复用。

2.5 协同仿真与测试平台

HDL 设计验证是数字系统开发中的关键环节。对于协议控制器而言，仅依靠编译通过不能说明功能正确，必须构造读写事务、边界情况、异常响应和时序变化等测试场景。协同仿真是一种常用方法，它允许用高级语言编写测试激励和行为模型，再与 Verilog 被测设计共同运行。

项目报告显示，`verilog-i2c-master` 使用 Python/MyHDL 行为模型和 Icarus Verilog 进行协同仿真。测试目录中包含 `axil.py`、`axis_ep.py`、`i2c.py` 和 `wb.py` 等总线模型文件，也包含多个 `test_*.py` 和对应 Verilog 测试顶层。Python 负责测试激励、总线行为和断言检查，Verilog 负责实际 DUT 实现。这种模式相比纯 Verilog testbench 更容易组织复杂测试逻辑，也便于在测试脚本中使用数据结构和流程控制。

3 系统需求分析

3.1 项目目标分析

根据项目报告，`verilog-i2c-master` 的项目目标是提供一组可复用的 I2C 主从控制器、总线包装层与初始化模块，用于在 Verilog 设计中快速集成 I2C 通信能力。该目标可以理解为三个层次。第一层是协议功能层，即实现 I2C 主模式和从模式基本通信能力。第二层是系统集成层，即将协议核心封装为可由 AXI-Lite、Wishbone 等片上总线访问的模块。第三层是工程支撑层，即提供初始化模板、简单从设备示例和测试平台，降低用户理解、移植和验证成本。

该项目的应用场景包括 FPGA 外设控制、上电初始化、片上总线与外部微控制器桥接、基于 I2C 的寄存器访问和数据交互。对于需要通过 FPGA 配置时钟芯片、传感器、电源管理芯片或其他低速外设的系统而言，I2C 主控制器是基础组件。对于需要让外部 MCU 通过 I2C 控制 FPGA 内部寄存器或片上总线资源的系统而言，I2C 从设备桥接模块具有实际意义。

3.2 功能需求分析

根据项目报告，项目实现的核心功能可以归纳如下。

第一，I2C 主控制功能。主控制器需要支持读、写、连续写、起始条件、重复起始条件和停止条件等操作。对于实际 I2C 通信而言，连续写和重复起始很重要，因为许多外设要求先写入寄存器地址，再通过重复起始发起读操作。如果控制器只支持单字节简单读写，就难以适配复杂外设。因此，项目将命令接口设计为可以表达多种事务类型，增强了通用性。

第二，I2C 从控制功能。从控制器需要响应主机访问，将写入数据转换为内部数据流输出，并将待发送数据通过 I2C 返回给主机。项目报告指出，`i2c_slave.v` 将 I2C 读写映射为 AXI Stream 数据收发，并支持地址匹配与时钟拉伸。这说明其目标不是固定功能的从设备，而是通用从设备协议引擎。

第三，片上总线包装功能。项目提供基于 AXI-Lite 的 I2C 主机包装层和基于 Wishbone 的 I2C 主机包装层，支持 8 位和 16 位从接口。包装层将协议控制器转换为寄存器可访问外设，使 CPU 或控制逻辑能够通过状态寄存器、命令寄存器、数据寄存器和分频寄存器进行控制。

第四，I2C 从设备桥接功能。项目提供 `i2c_slave_axil_master.v` 和 `i2c_slave_wbm.v`，可把外部 I2C 请求转换为 AXI-Lite 或 Wishbone 主控访问。这种设计允许外部 MCU 通过 I2C 总线间接访问 FPGA 内部寄存器或存储空间，适合低速控制、调试和状态读取场景。

第五，上电初始化功能。项目提供 `i2c_init.v`，通过 ROM 指令表自动驱动一系列 I2C 命令和数据输出，适合在系统启动时配置时钟芯片、PLL、抖动清理器、复用器或其他外设。相比由 CPU 软件完成初始化，硬件初始化状态机可以在无处理器系统中独立工作。

第六，示例与测试功能。项目提供 `i2c_single_reg.v` 作为简化单寄存器 I2C 从设备示例，并提供 MyHDL 测试平台，用于验证主机、从机、初始化模块和总线包装层等功能。

从毕业设计的角度看，上述功能需求之间并不是彼此孤立的。I2C 主控制器解决 FPGA 主动访问外部器件的问题，I2C 从控制器解决 FPGA 被外部控制器访问的问题，片上总线包装层解决 FPGA 内部控制逻辑或软核处理器如何统一管理 I2C 访问的问题，初始化模块解决无处理器或低软件参与场景下的自动配置问题，测试平台则为这些模块能否正常工作提供验证支撑。因此，该项目并非单一协议状态机，而是围绕 I2C 通信形成了较完整的接口组件集合。

在需求优先级上，核心主从控制器应处于最高层级。如果主从控制器无法稳定产生或响应 I2C 时序，其他包装层和桥接层都无法发挥作用。其次是总线适配层，因为在实际 FPGA 系统中，外设接口往往需要通过片上总线进行管理。再次是初始化与示例模块，它们提升了项目在具体场景中的易用性。最后是测试平台，虽然测试平台不直接参与硬件部署，但它决定了项目是否具备可验证、可维护和可扩展的工程基础。

3.3 非功能需求分析

除功能需求外，项目还体现出多项非功能需求。

第一，可移植性。项目使用 Verilog 2001 编写，便于在常见 FPGA 工具链中复用。相较于依赖特定厂商 IP 的方案，纯 Verilog 组件更易迁移到不同 FPGA 平台。但实际迁移仍需要根据目标器件补充约束、顶层封装和板级验证。

第二，可配置性。项目包含分频参数、输入滤波长度、FIFO 深度、数据宽度、地址宽度和设备地址掩码等配置项。可配置性使同一模块能够适配不同 I2C 速率、不同片上总线位宽和不同从设备地址范围。

第三，接口规范性。项目使用 AXI Stream、AXI-Lite 和 Wishbone 等常见接口风格，而不是自定义孤立接口。这有助于与现有 FPGA 系统、软核 CPU 和测试模型集成。

第四，可验证性。项目包含 MyHDL 协同仿真环境，说明其设计目标不仅是提供 RTL 文件，还包括提供验证支撑。对于接口协议组件而言，测试平台是评估可靠性的重要依据。

第五，安全和约束意识。项目文档明确说明 I2C 引脚采用开漏或三态连接方式，不能将 `scl_o` 直接连接到 `scl_i`。这一点非常重要，因为 I2C 的物理接口模型与普通推挽输出不同。如果连接方式错误，仿真和硬件行为都可能出现异常。

3.4 设计约束与问题

项目报告中还指出了若干约束和已知问题。首先，项目测试依赖 MyHDL、Icarus Verilog 和 `myhdl.vpi` 协同仿真支持。如果用户环境未正确安装这些依赖，就无法直接运行测试脚本。其次，AXI-Lite 和 Wishbone 封装对位宽有参数化约束，源码中包含宽度合法性断言，说明设计对总线宽度组合有一定限制。再次，README 明确说明该仓库已弃用，后续不再维护，新特性与修复迁移到其他仓库。因此，若将其用于新项目，需要评估后继仓库或自行维护风险。

此外，当前项目报告未提供综合结果、时序收敛报告、资源利用率报告、特定 FPGA 板卡顶层工程和约束文件，也未提供实际运行测试的日志。因此，论文在描述验证结果时只能说明“项目具有测试平台和测试设计”，不能直接声称“所有测试已通过”或“硬件验证成功”。

3.5 可行性分析

从技术可行性看，该项目采用 Verilog 2001 编写，并围绕 I2C 这种成熟低速总线协议展开，技术路线明确。I2C 协议本身具有公开资料和大量应用案例，FPGA 工具链也普遍支持 Verilog RTL 综合，因此核心协议控制器在技术上具备可实现基础。项目中使用的 AXI Stream、AXI-Lite 和 Wishbone 接口均是常见数字系统接口形式，相关设计思想成熟，便于与已有系统连接。

从工程可行性看，项目已形成较完整目录结构和模块集合。`rtl/` 目录包含核心可综合代码，`tb/` 目录包含测试模型和测试脚本，README 对项目用途和测试依赖进行了说明。这说明项目已经具备被阅读、仿真和二次移植的基本条件。对于本科毕业设计而言，即使不进行完整板级部署，也可以围绕模块结构、接口关系、仿真测试和工程复用价值展开较完整研究。

从验证可行性看，项目采用 MyHDL 与 Icarus Verilog 的协同仿真方式。Icarus Verilog 是常用开源 Verilog 仿真工具，MyHDL 可用 Python 编写行为模型和测试激励，二者结合可以降低复杂总线模型的编写难度。项目报告已经指出测试脚本不仅进行编译检查，还包含对命令输出、地址序列、数据内容和结束条件的断言。因此，只要环境依赖配置正确，后续具备进一步运行测试并补充实验结果的条件。

从风险角度看，该项目最大的限制是仓库已弃用且缺少实际运行结果。对于毕业论文写作，这要求本文在表述中区分项目已有内容与后续待验证内容；对于实际工程使用，则要求使用者评估后继仓库或自行补充维护。总体而言，该项目适合作为 HDL 接口组件库分析、学习和二次开发基础，作为直接量产级 IP 使用则仍需进一步验证。

4 系统总体设计

4.1 总体架构

`verilog-i2c-master` 项目采用分层架构，可概括为核心层、扩展层、应用层、支撑层和验证层。

核心层由 `i2c_master.v` 和 `i2c_slave.v` 构成，负责实现 I2C 主机和从机基本协议行为。它们直接处理 I2C 时序、地址、读写、应答和状态反馈，是整个项目的基础。

扩展层由 `i2c_master_axil.v`、`i2c_master_wbs_8.v`、`i2c_master_wbs_16.v`、`i2c_slave_axil_master.v` 和 `i2c_slave_wbm.v` 构成。该层将核心协议能力与片上总线系统连接起来。一方面，主机包装层把 I2C 主控制器暴露为 AXI-Lite 或 Wishbone 从设备；另一方面，从机桥接层把外部 I2C 主设备的访问转换为内部 AXI-Lite 或 Wishbone 主控事务。

应用层由 `i2c_init.v` 和 `i2c_single_reg.v` 构成。前者提供上电初始化状态机模板，可用于批量配置外设；后者提供简化单寄存器从设备示例，适合教学和快速验证。

支撑层包括 `axis_fifo.v`。该模块是通用 AXI4-Stream FIFO，用于数据缓冲和流式接口解耦。通过 FIFO，命令和数据可以在协议状态机和上层接口之间平滑传递，降低时序耦合。

验证层位于 `tb/` 目录，包含 AXI-Lite、AXI Stream、I2C、Wishbone 等行为模型以及多个测试脚本。验证层使项目能够在仿真环境中构造主从交互、数据读写和初始化序列等测试场景。

4.2 工程目录结构

项目报告给出的工程结构如下：

```
verilog-i2c-master/  
  README.md  
  rtl/  
    axis_fifo.v  
    i2c_init.v  
    i2c_master.v  
    i2c_master_axil.v  
    i2c_master_wbs_8.v  
    i2c_master_wbs_16.v  
    i2c_single_reg.v  
    i2c_slave.v  
    i2c_slave_axil_master.v  
    i2c_slave_wbm.v  
  tb/  
    axil.py
```

```
axis_ep.py
i2c.py
wb.py
test_i2c*.py
test_i2c*.v
```

从该目录结构可以看出，项目将可综合 RTL 文件集中放置在 `rtl/` 目录，将测试平台文件集中放置在 `tb/` 目录。这种结构符合开源 HDL 项目的常见组织方式，有利于用户快速定位设计文件和验证文件。`README.md` 则提供模块说明、接口信息和测试方式，是用户理解项目的入口。

这种目录组织方式也体现了设计与验证分离的工程习惯。对于 HDL 项目而言，可综合代码和测试代码在语义上具有明显区别。前者需要经过综合、布局布线并最终映射到 FPGA 资源，后者主要服务于仿真验证，不进入最终硬件。将二者分开放置可以减少误用风险，也便于自动化脚本分别处理 RTL 文件和测试文件。同时，项目没有把所有功能堆叠在单个模块中，而是把主机、从机、包装层、桥接层和示例模块拆分为独立文件，有利于用户按需选取。

从复用角度看，`rtl/` 中每个模块都对应较明确的应用场景。用户如果只需要最基础的 I2C 主机，可以直接集成 `i2c_master.v`；如果已有 AXI-Lite 控制总线，则可以选择 `i2c_master_axil.v`；如果系统使用 Wishbone，则可以选择对应 Wishbone 版本；如果需要外部 MCU 通过 I2C 访问 FPGA 内部寄存器，则可以使用从机桥接模块。这种按功能拆分的组织形式降低了项目集成成本。

4.3 数据流设计

4.3.1 主机数据流 I2C 主机侧的数据流可以概括为：上层逻辑写入命令和数据，I2C 主控制器解析命令并驱动总线，读回数据再返回上层逻辑。具体过程包括以下步骤。

首先，上层逻辑通过 AXI Stream 或寄存器接口提交命令。命令可以表示起始、读、写、连续写或停止等操作。如果使用 AXI-Lite 或 Wishbone 包装层，则命令由 CPU 或控制器写入命令寄存器；如果直接使用核心模块，则命令通过流式接口输入。

其次，控制器将命令转换为 I2C 总线时序。内部命令状态机负责决定当前执行地址阶段、读阶段、写阶段还是停止阶段；底层 PHY 状态机负责在 SCL 和 SDA 上产生符合协议的位级变化。

再次，若执行写操作，待发送数据通过写数据通道进入控制器，并在 SCL 时钟控制下逐位输出；若执行读操作，从设备返回的数据被采样并通过输出数据通道返回。

最后，控制器通过状态信号反馈执行情况。例如，当从设备未应答时，`missed_ack` 可用于提示上层逻辑通信失败。若总线仍处于连续访问状态，相关状态也可反映当前总线是否忙碌或保持控制。

4.3.2 从机数据流 I2C 从机侧的数据流以外部主设备发起访问为起点。外部主设备首先发送从设备地址和读写位，从控制器检测总线并判断地址是否匹配。如果地址匹配，从控制器返回 ACK 并进入读或写事务。

在写事务中，外部主设备发送的数据字节被从控制器接收，并转换为内部 AXI Stream 输出。这样，上层逻辑可以通过流式接口读取外部写入的数据。在读事务中，从控制器从 AXI Stream 输入端获取待发送数据，并在 I2C 主设备提供的时钟下发送给主设备。如果待发送数据尚未准备好，从控制器可以通过时钟拉伸保持 SCL 为低，从而等待内部数据就绪。

该设计使 I2C 从控制器不绑定具体寄存器模型，而是提供通用数据收发能力。用户可以在其外部连接寄存器文件、FIFO、片上总线桥接逻辑或其他控制模块。

4.3.3 I2C 到片上总线桥接数据流 项目提供的从机桥接层进一步扩展了 I2C 从设备能力。以 `i2c_slave_axil_master.v` 和 `i2c_slave_wbm.v` 为例，外部 MCU 可以通过 I2C 写入目标地址，然后继续写入数据或发起读操作。模块在内部保存地址指针，并将后续事务转换为 AXI-Lite 或 Wishbone 总线访问。

这种桥接流程可以用于低速调试和控制。例如，外部 MCU 不需要直接连接 FPGA 内部总线，只需通过 I2C 即可访问部分寄存器空间。由于地址指针支持自动递增，连续寄存器读写也更方便。对于数据宽度超过 8 位的片上总线，桥接模块还需要处理对齐、补零、合并和拆分等问题，项目报告指出 AXI-Lite 和 Wishbone 桥接模块均包含类似处理逻辑。

4.4 控制流设计

项目的控制流核心是状态机。I2C 主控制器内部可理解为两级状态机：较高层的命令状态机负责处理事务级操作，如发起开始条件、发送地址、执行读写、生成停止条件等；较底层的 PHY 状态机负责位级时序，如 SCL

拉低、SDA 设置、SCL 拉高、数据采样和 ACK 检测等。两级状态机分离有利于降低设计复杂度，使事务控制和物理时序控制职责清晰。

I2C 从控制器的控制流则以总线监听为起点。模块需要检测起始条件，接收地址，判断地址匹配，返回应答，然后根据读写位进入接收或发送状态。由于从设备由外部主机提供时钟，它还需要在输入数据、输出数据和时钟拉伸之间保持协调。

初始化模块 `i2c_init.v` 的控制流则基于 ROM 指令表。模块在 `start` 拉起后，按顺序读取初始化指令，将其转换为 I2C 主控命令和数据流。指令表可包含写入数据和延时命令，从而描述多设备、多步骤的初始化过程。这种方法把固定初始化流程从复杂状态机中抽象为指令序列，提高了可配置性和可读性。

4.5 接口设计原则

该项目的接口设计体现出“协议核心简单化、系统集成标准化”的原则。核心 I2C 模块采用接近协议语义的信号和 AXI Stream 风格数据通道，使其既能被上层包装层驱动，也能直接被自定义逻辑使用。包装层则采用 AXI-Lite 和 Wishbone 等标准片上总线接口，使控制器能够被软核 CPU 或系统控制器统一管理。

I2C 物理接口采用 `scl_i`、`scl_o`、`scl_t` 和 `sda_i`、`sda_o`、`sda_t` 的形式。这种输入、输出、三态控制分离的接口形式适合 FPGA I/O 单元建模，也符合 I2C 开漏连接特点。由于 I2C 总线高电平通常由上拉电阻提供，设备只主动拉低总线或释放总线，因此不能像普通推挽信号一样简单将输出反馈到输入。项目报告中特别指出不能将 `scl_o` 直接连接到 `scl_i`，说明设计者对物理层连接约束有明确提示。

在内部数据接口上，AXI Stream 风格握手可以提高模块之间的解耦程度。I2C 总线速率通常远低于系统时钟，如果上层逻辑必须等待每一个 I2C 位级动作完成，会造成控制复杂且效率较低。通过命令流、写数据流和读数据流，上层逻辑可以以事务为单位提交操作，协议模块则在内部逐步执行。这种设计也为 FIFO 缓冲和寄存器包装提供了基础。

在片上总线接口上，AXI-Lite 和 Wishbone 的引入使项目可以适配不同系统生态。AXI-Lite 适合较新的 SoC 和 FPGA IP 集成环境，Wishbone 则在开源软核和传统开源 HDL 项目中常见。项目同时提供两类接口，增强了组件库的适用范围。

5 详细设计与实现分析

5.1 I2C 主控制器设计

`i2c_master.v` 是项目的核心模块之一，用于实现 I2C 主模式通信。根据项目报告，该模块命令接口支持 `read`、`write`、`write multiple` 和 `stop` 等操作。其关键特性包括支持显式起始和停止，也支持根据总线状态隐式起始；通过 `prescale` 参数进行分频；提供 `misted_ack` 状态输出；内部分为较高层命令状态机和底层 PHY 状态机。

I2C 主控制器的设计难点在于将上层抽象命令可靠转换为总线上的时序变化。上层逻辑希望用“读一个字节”“写多个字节”“发送停止”等命令描述事务，而 I2C 总线实际要求控制器逐位控制 SDA 和 SCL，并在每个字节之后检测或生成 ACK。因此，分层状态机设计是合理的。命令状态机面向事务语义，决定下一步应该发送地址、发送数据、接收数据还是停止；PHY 状态机面向物理时序，负责在正确时刻改变或采样信号。

`prescale` 分频参数用于生成 I2C 时钟。项目报告指出源码中给出公式 $\text{prescale} = \text{Fclk} / (\text{FI2Cclk} * 4)$ 。这说明模块内部可能将一个 I2C 时钟周期划分为若干阶段，用于控制 SCL 低电平、高电平以及数据建立和保持时间。通过分频参数，用户可以在不同系统时钟频率下生成目标 I2C 速率。

`misted_ack` 是 I2C 主机设计中的重要状态。当主设备发送地址或数据后，从设备应在第九个时钟周期拉低 SDA 表示应答。如果未检测到应答，控制器需要向上层报告异常。该状态可以帮助软件或控制逻辑判断外设是否存在、总线是否正常、地址是否正确或设备是否忙碌。

5.2 I2C 从控制器设计

`i2c_slave.v` 用于实现 I2C 从模式通信。项目报告指出，该模块支持 I2C 读写事务解析，将写入的字节转换为输出 AXI Stream 数据，将待发送数据从输入 AXI Stream 读取后返回给 I2C 主机，并支持时钟拉伸。此外，模块提供 `busy`、`bus_address`、`bus_addressed`、`bus_active` 等状态信号，以及 `device_address` 和 `device_address_mask` 用于设备地址匹配。

I2C 从控制器的设计与主控制器不同。主控制器主动产生时钟和事务，而从控制器必须被动监听总线。它需要在检测到起始条件后开始接收地址位，并根据设备地址和掩码判断是否响应。地址掩码机制有利于支持多个地址或地址范围匹配，使设计更灵活。

在写事务中，从控制器接收外部主机发送的数据字节，并通过输出 AXI Stream 通道交给内部逻辑。在读事务中，从控制器需要从输入 AXI Stream 通道获得待发送数据。如果上层数据尚未准备好，模块可以通过拉低 SCL 实现时钟拉伸。时钟拉伸是 I2C 协议允许从设备暂停主机时钟的一种机制，对于内部处理速度慢于总线访问速度的设备非常有用。

状态信号的设计也很重要。`busy` 可表示模块正在处理事务，`bus_active` 可表示总线处于活动状态，`bus_addressed` 可表示当前从设备被主机选中，`bus_address` 可输出当前地址信息。这些状态有助于上层逻辑进行调试、统计和控制。

5.3 AXI-Lite 主机包装层设计

`i2c_master_axil.v` 将 I2C 主控制器封装为 AXI-Lite 从设备。根据项目报告，该模块提供 4 个主要寄存器：状态、命令、数据、分频。状态寄存器包含 `busy`、`bus_cont`、`bus_act`、`miss_ack` 以及 FIFO 状态；命令寄存器允许组合 `start`、`read`、`write`、`write multiple` 和 `stop`；数据寄存器既可写入待发送字节，也可读取接收字节；模块还提供可配置的命令 FIFO、写 FIFO 和读 FIFO 深度。

AXI-Lite 包装层的价值在于使 I2C 控制器可以被 CPU 或软核处理器以寄存器方式访问。对于 SoC 风格 FPGA 系统，软件通常通过读写外设寄存器控制硬件模块。如果只有 AXI Stream 命令接口，软件访问并不方便；而 AXI-Lite 包装层提供了标准总线接口，使控制流程可以由软件驱动。

FIFO 的引入有助于提高控制器与软件之间的解耦程度。软件可以连续写入多个命令或数据，控制器按协议时序逐步执行；读回数据也可以先进入读 FIFO，再由软件读取。这样可以降低软件轮询频率对总线时序的影响。对于低速 I2C 总线而言，FIFO 深度不一定需要很大，但可配置 FIFO 提供了更好的适配能力。

5.4 Wishbone 主机包装层设计

项目提供 `i2c_master_wbs_8.v` 和 `i2c_master_wbs_16.v` 两个 Wishbone 主机包装层。根据项目报告，`i2c_master_wbs_8.v` 将状态、FIFO 状态、命令地址、命令、数据和分频映射到不同地址，适合基于 Wishbone 的软核系统或开源总线架构。`i2c_master_wbs_16.v` 可以合理理解为采用相近思路，但接口和寄存器宽度适配为 16 位。

Wishbone 总线在开源 FPGA 项目中具有较长历史，许多软核和外设 IP 使用该接口。项目同时提供 AXI-Lite 和 Wishbone 封装，说明其目标用户不局限于某一种片上总线生态。对于组件库而言，多总线适配可以扩大复用范围，也有助于用户在已有系统中直接集成。

8 位和 16 位版本的区别可能体现在数据总线宽度、地址映射和寄存器访问方式上。由于 I2C 数据本身以字节为基本单位，8 位接口具有自然匹配优势；16 位接口则更适合某些软核或系统总线配置。无论哪种版本，包装层的核心任务都是把总线读写转换为 I2C 控制器可理解的命令、数据和配置操作。

5.5 I2C 从设备桥接层设计

`i2c_slave_axil_master.v` 和 `i2c_slave_wbm.v` 是项目中较有特色的模块。它们不是简单的 I2C 从设备，而是将外部 I2C 主机访问桥接到 FPGA 内部片上总线。根据项目报告，`i2c_slave_axil_master.v` 中，I2C 写操作先访问内部地址寄存器，再写 AXI-Lite 地址空间；I2C 读操作从当前内部地址开始发起 AXI-Lite 读；对于宽于 8 位的总线，模块处理对齐、补零和同地址数据合并；地址指针会自动递增。

这种设计使 FPGA 可以作为 I2C 外设挂接到外部控制器上。外部 MCU 只需要通过 I2C 写入地址和数据，就可以访问 FPGA 内部寄存器。这在系统调试、板级控制、状态读取和低速管理场景中非常有用。例如，一个 FPGA 系统可能内部包含多个控制寄存器，但外部主控制器只提供 I2C 接口。通过桥接模块，可以在不增加复杂外部接口的情况下实现控制访问。

`i2c_slave_wbm.v` 与 AXI-Lite 版本类似，但桥接到 Wishbone 主接口。项目报告指出，该模块保留了参数化数据宽度、地址宽度、选择信号宽度，对位宽和字宽进行合法性断言，并具有地址寄存器、自动递增和读写合并语义。这说明桥接模块不仅实现协议转换，还考虑了不同总线数据宽度带来的访问粒度问题。

5.6 初始化模块设计

`i2c_init.v` 是一个模板化初始化状态机。根据项目报告，该模块通过 `init_data` ROM 存储 9 位宽初始化指令，支持单设备初始化和多设备初始化两种模式，可插入延时命令；`start` 拉起后，模块自动按表驱动输出 I2C 主控命令与数据流。

在实际 FPGA 系统中，许多外设需要在上电后写入一系列配置寄存器。例如时钟芯片需要设置频率、输出通道和锁相环参数；图像传感器需要配置分辨率、曝光和数据格式；电源管理芯片需要配置电压轨和时序。如果系统没有嵌入式 CPU，或者希望在 CPU 启动前完成部分硬件初始化，就可以使用硬件初始化状态机。

将初始化流程编码为 ROM 指令表具有明显优势。第一，初始化数据与状态机控制逻辑分离，便于修改配置而不改动核心控制流程。第二，多个设备初始化可以按表顺序执行，结构清晰。第三，延时命令可以处理某些外设需要等待稳定时间的场景。第四，模块输出 I2C 主控命令与数据流，可复用主控制器完成实际总线时序。

5.7 单寄存器从设备示例

`i2c_single_reg.v` 是简化的 I2C 从设备实现。项目报告指出，该模块固定设备地址 `DEV_ADDR`，只提供单字节寄存器读写，接口简单，适合教学、快速验证或构建最小化外设示例。

虽然该模块功能简单，但在组件库中具有示例价值。对于初学者而言，直接理解通用从控制器和片上总线桥接模块可能较难；单寄存器从设备则提供了一个最小可理解实例，展示 I2C 地址匹配、写入寄存器和读回寄存器的基本流程。对于工程验证而言，该模块也可作为测试主控制器读写行为的简单目标。

5.8 AXI Stream FIFO 设计

`axis_fifo.v` 是通用 AXI4-Stream FIFO。项目报告指出，该模块参数化深度、数据宽度、`tkeep`、`tlast`、`tuser` 等，支持帧模式和帧丢弃机制，在 I2C 项目中主要承担缓冲和解耦作用。

在 I2C 控制器中，协议时序通常比片上总线或内部逻辑慢得多。如果上层逻辑和 I2C 状态机直接耦合，可能导致接口控制复杂。通过 FIFO，命令、写数据和读数据可以缓存起来，使生产者和消费者在一定范围内独立运行。对于 AXI-Lite 包装层，FIFO 还能减少软件访问时序与 I2C 执行时序之间的直接依赖。

从工程设计角度看，FIFO 的意义不仅在于缓存数据，还在于吸收不同模块之间的速率差异。AXI-Lite 或 Wishbone 侧的控制逻辑可能在系统时钟下快速写入命令，而 I2C 总线受分频参数和协议时序限制，执行速度明显较慢。若没有缓冲机制，上层控制逻辑必须频繁等待底层协议完成，系统吞吐和软件控制体验都会受到影响。通过可配置 FIFO，模块可以在一定范围内连续接收命令或数据，再由协议状态机逐项执行。

此外，FIFO 还有助于提高模块边界清晰度。上层包装层只需要关心 FIFO 是否满、是否空以及何时读写数据，而底层 I2C 控制器只需要按照握手信号获取命令和数据。两者之间不必共享复杂内部状态。这种边界清晰的设计方式符合可复用 IP 的基本要求。

5.9 关键参数与配置分析

项目报告列出了多个关键参数，包括 `prescale`、`FILTER_LEN`、`CMD_FIFO_DEPTH`、`WRITE_FIFO_DEPTH`、`READ_FIFO_DEPTH`、`DATA_WIDTH`、`ADDR_WIDTH`、`STRB_WIDTH`、`WB_DATA_WIDTH`、`WB_ADDR_WIDTH`、`WB_SELECT_WIDTH`、`device_address` 和 `device_address_mask` 等。这些参数覆盖了时钟、滤波、缓冲、片上总线位宽和从设备地址匹配等多个方面。

`prescale` 是 I2C 主控制器中最直接影响通信速率的参数。由于 FPGA 系统时钟通常远高于 I2C 总线速率，控制器需要通过分频产生合适的 SCL 时序。项目报告中给出的公式表明，设计者将 I2C 一个周期划分为多个内部阶段，以便控制数据建立、采样和保持过程。用户在移植模块时，应根据系统时钟频率和目标 I2C 速率重新计算该参数。

`FILTER_LEN` 与输入滤波有关。I2C 总线可能受到毛刺、上升沿缓慢或外部干扰影响，输入滤波可以提高信号判断稳定性。滤波长度过短可能无法过滤噪声，过长则可能影响边沿响应速度，因此应结合目标总线速率和硬件环境进行配置。

FIFO 深度参数决定了命令和数据缓冲能力。较大的 FIFO 可以容纳更多未执行命令或待读数据，提高突发访问场景下的缓冲能力，但也会增加资源消耗。对于低速控制场景，较小 FIFO 可能已经足够；对于需要连续读写多个寄存器的外设配置场景，适当增加 FIFO 深度可以减少上层等待。

总线位宽参数影响 AXI-Lite 和 Wishbone 包装层的地址映射和数据对齐。对于 I2C 这种以字节为基本传输单位的协议，当片上总线宽度大于 8 位时，桥接模块需要处理字节选择、地址递增、数据合并和补零等问题。项目报告指出相关模块中存在位宽合法性断言，说明设计对参数组合进行了一定约束。

`device_address` 和 `device_address_mask` 则用于 I2C 从设备地址匹配。通过掩码机制，一个模块可以响应特定地址或地址范围，增强配置灵活性。但在实际系统中，地址配置必须避免与总线上其他设备冲突，否则可能出现多个从设备同时应答的问题。

5.10 实现小结

综合上述分析，`verilog-i2c-master` 的实现思路可以概括为：以 I2C 主从协议状态机为核心，以 AXI Stream 风格接口实现命令与数据解耦，以 AXI-Lite 和 Wishbone 包装层面向系统集成，以初始化和单寄存

器示例面向具体应用，以测试平台支撑功能验证。该实现方式既体现了 HDL 模块化设计思想，也体现了开源接口组件库的工程组织方式。

需要指出的是，本文的实现分析主要依据项目报告，而不是重新逐行审查所有源码。因此，对具体状态机编码、寄存器位定义和边界条件的进一步确认，还需要在后续工作中结合源码、仿真波形和测试日志展开。

6 测试与验证分析

6.1 测试平台组成

项目报告显示，`verilog-i2c-master` 包含较完整的测试体系。`tb/axil.py` 提供 AXI4-Lite 主机与内存行为模型；`tb/axis_ep.py` 提供 AXI Stream 端点模型；`tb/i2c.py` 提供 I2C 主从模型；`tb/wb.py` 提供 Wishbone 主机与 RAM 模型；多个 `test_*.py` 对应不同 RTL 模块的专项测试。

这种测试组织方式体现了分层验证思想。对于主控制器，可以使用 I2C 存储器模型作为被访问对象，测试不同地址、延迟和命令/数据流。对于从控制器，可以使用 I2C 主机模型驱动 DUT，检查其地址响应、数据接收、数据发送和时钟拉伸行为。对于初始化模块，可以检查其发出的地址与数据序列是否符合预期。对于总线包装层，可以通过 AXI-Lite 或 Wishbone 行为模型模拟软件或片上总线访问。

6.2 协同仿真流程

根据项目报告，项目测试依赖 MyHDL、Icarus Verilog 和 `myhdl.vpi`。典型流程包括安装 MyHDL、安装 Icarus Verilog、确保 `myhdl.vpi` 正确安装，然后使用 `py.test`、`nose` 或直接运行 Python 测试脚本。测试脚本内部会调用类似 `iverilog -o test_xxx.vvp ...` 的命令编译 Verilog 文件，再使用 `vvp -m myhdl` 运行协同仿真。

该流程中，Python 测试脚本既负责生成激励，也负责检查结果。Verilog 被测模块作为 DUT 参与仿真。MyHDL 提供了 Python 与 Verilog 仿真之间的连接机制，使 Python 行为模型能够与 HDL 信号交互。相较于手写纯 Verilog testbench，这种方法更容易构造复杂行为模型，例如 I2C 存储器、AXI-Lite 内存和 Wishbone RAM。

6.3 验证覆盖内容

从项目报告可知，测试脚本覆盖了多个方面。`test_i2c_master.py` 构造了两个 I2C 存储器模型，并测试不同地址、延迟和命令/数据流；`test_i2c_slave.py` 使用 I2C 主机模型驱动 DUT，并通过总线线与逻辑模拟开漏连接；`test_i2c_init.py` 检查初始化模块发出的地址与数据序列是否符合预期。

这些测试内容说明项目验证并非只检查编译是否通过，而是针对协议行为进行功能验证。对于 I2C 主机，测试需要关注是否正确产生地址、读写位、数据字节和停止条件，是否能够接收数据并检测应答。对于 I2C 从机，测试需要关注地址匹配、数据接收、数据发送和时钟拉伸。对于初始化模块，测试需要关注指令表是否被正确解释，输出序列是否符合预设。

6.4 验证结果与谨慎表述

需要特别说明的是，本文生成依据为项目报告，而项目报告明确指出本次任务中未实际执行仿真。因此，本文不能将“测试已全部通过”写作确定事实。基于项目报告，可以得出的谨慎结论是：项目作者为各主要模块提供了配套测试，测试结构包含对命令输出、地址序列、数据内容和结束条件的断言，验证设计覆盖了主机、从机、初始化流程以及不同总线桥接场景。

换言之，项目具备较完整的仿真验证设计和测试平台，但当前论文材料不足以证明这些测试在本地环境中已经成功运行。若后续需要形成更完整的毕业论文成果，应实际搭建依赖环境并运行测试，保存测试日志、波形截图或断言结果，再将其作为第六章的正式实验结果。

6.5 测试不足与改进方向

从项目报告看，当前材料未提供综合结果、时序收敛报告和资源利用率数据，也未提供特定 FPGA 板卡的顶层工程和约束文件。因此，该项目验证仍主要停留在仿真层面。仿真可以证明功能行为在测试场景下符合预期，但不能替代综合后时序分析和硬件上板验证。

后续可以从以下方向补充。第一，运行现有 MyHDL 测试，记录测试命令、通过结果和关键波形。第二，选择一个目标 FPGA 器件进行综合，记录 LUT、触发器、块 RAM 等资源使用情况。第三，在不同 I2C 分频参数下进行时序分析，确认最高可运行频率和约束满足情况。第四，构建板级示例工程，连接真实 I2C 外设进行读写验证。第五，引入更现代的 cocotb 或 SystemVerilog 验证方法，提高测试可维护性和覆盖率。

7 总结与展望

7.1 全文总结

本文基于 `project-report.md` 对 `verilog-i2c-master` 项目进行了本科毕业论文形式的整理和分析。该项目是一个面向 FPGA/HDL 开发的 I2C 接口组件库，提供 I2C 主从控制器、AXI-Lite 和 Wishbone 包装层、I2C 到片上总线桥接模块、初始化状态机、单寄存器从设备示例以及 MyHDL 协同仿真测试平台。

通过需求分析可以看出，该项目面向 FPGA 外设控制、上电初始化、片上总线桥接和寄存器访问等典型场景，功能需求较完整，非功能需求体现出可移植性、可配置性、接口规范性和可验证性。通过总体设计分析可以看出，项目采用分层结构，将核心协议引擎、总线适配、应用示例和测试支撑分离，有利于模块复用和系统集成。通过详细设计分析可以看出，主控制器采用事务级命令状态机与位级 PHY 状态机分离的思路，从控制器通过 AXI Stream 接口与上层逻辑解耦，总线包装层和桥接层则使 I2C 协议能力能够嵌入更复杂的片上系统。

在测试与验证方面，项目提供了较完整的 MyHDL 协同仿真环境，包括 AXI-Lite、AXI Stream、I2C 和 Wishbone 行为模型。测试脚本针对主机、从机和初始化模块设计了功能验证场景。由于当前材料未实际执行仿真，本文仅将其表述为“具备测试平台和验证设计”，不将测试通过作为事实结论。

7.2 项目优势

该项目的主要优势包括以下几点。

第一，模块划分清晰。核心协议模块、总线包装模块、应用示例模块和测试平台边界明确，便于理解和复用。

第二，接口适配能力较强。项目不仅提供基础 I2C 主从控制器，还提供 AXI-Lite 和 Wishbone 适配层，能够适应不同片上系统环境。

第三，参数化程度较高。项目支持分频、滤波长度、FIFO 深度、数据位宽、地址位宽和设备地址掩码等配置，有利于在不同系统中移植。

第四，验证支撑较完整。项目包含 MyHDL 协同仿真测试平台，测试文件不只是形式化存在，而是围绕主机、从机和初始化模块构造了具体测试场景。

第五，教学价值较明显。项目展示了从基础协议控制器到系统级总线接口封装的完整过程，适合用于学习 HDL 模块化设计和接口组件库构建方法。

7.3 项目不足

项目也存在一些不足。首先，README 已明确说明该仓库已弃用，后续不再维护。这意味着在新项目中直接使用该仓库需要承担维护风险。其次，当前材料中缺少综合报告、时序报告、资源利用率报告和 FPGA 板级验证结果，因此无法评估该设计在具体器件上的资源开销和时序性能。再次，项目没有提供特定开发板的顶层工程、引脚约束和真实外设演示，用户需要自行完成上板集成。最后，测试平台依赖 MyHDL 和 Icarus Verilog，对初学者而言可能存在环境搭建门槛。

7.4 后续展望

后续工作可以围绕工程化、验证完善和文档增强三个方向展开。

在工程化方面，可以选择常见 FPGA 开发板构建示例工程，补充约束文件、顶层连接和外设读写演示。也可以根据目标器件生成综合报告和时序报告，比较不同 FIFO 深度、不同总线包装层和不同分频参数下的资源使用情况。

在验证完善方面，可以实际运行现有测试脚本，保存测试日志和波形文件；也可以引入 cocotb 或 SystemVerilog 验证框架，构建更现代的回归测试环境。对于 I2C 协议，还可以增加异常场景测试，例如从设备无应答、总线忙、时钟拉伸超时、连续读写边界和不同地址掩码匹配。

在文档增强方面，可以补充模块接口时序图、寄存器映射表、典型使用流程和移植指南。对于 AXI-Lite 和 Wishbone 包装层，应给出软件访问示例；对于 `i2c_init.v`，应给出初始化指令表的配置方法和实际外设配置案例。

综上所述，`verilog-i2c-master` 虽然已经停止维护，但其模块结构、接口封装和测试组织仍具有较高参考价值。通过该项目进行系统分析，可以帮助理解 FPGA I2C 接口组件库的设计方法，也为后续构建更完整、更现代的 HDL 外设接口库提供基础。

致谢

在本文整理过程中，项目报告 `project-report.md` 为论文提供了主要事实依据，包括项目目标、模块结构、功能需求、实现内容和测试平台说明。感谢开源项目作者提供 Verilog I2C 接口组件库，使学习者能够通过真实工程案例理解 I2C 协议控制器、片上总线封装和协同仿真测试方法。感谢指导教师在毕业设计选题、论文结构和技术表达方面给予的指导。由于当前材料未包含实际仿真运行日志、综合报告和板级测试结果，本文在相关结论中采用谨慎表述，后续仍需通过实际实验进一步完善。

参考文献

注：以下参考文献主要依据项目材料和通用技术来源整理。正式提交论文前，建议进一步通过 Google Scholar、CNKI 或学校图书馆补充 I2C 协议、FPGA 接口设计、AXI/Wishbone 总线和 HDL 验证相关学术文献，并按 GB/T 7714—2015 核对格式。

- [1] Philips Semiconductors. The I2C-bus specification and user manual[S].
- [2] ARM. AMBA AXI and ACE Protocol Specification[S].
- [3] OpenCores. Wishbone B4 WISHBONE System-on-Chip Interconnection Architecture for Portable IP Cores[S].
- [4] IEEE. IEEE Standard for Verilog Hardware Description Language[S].
- [5] `verilog-i2c-master` 项目 README 与 RTL 源码材料 [EB/OL].
- [6] `project-report.md`: 项目规划设计与详细内容总结 [Z].

附录 A 项目主要模块清单

模块	功能说明
<code>i2c_master.v</code>	I2C 主控制器，负责主模式读写、起始、停止、ACK 检测和时序控制
<code>i2c_slave.v</code>	I2C 从控制器，负责地址匹配、数据接收、数据发送和时钟拉伸
<code>i2c_master_axil.v</code>	AXI-Lite 从接口包装层，通过寄存器控制 I2C 主机
<code>i2c_master_wbs_8.v</code>	8 位 Wishbone 从接口包装层
<code>i2c_master_wbs_16.v</code>	16 位 Wishbone 从接口包装层
<code>i2c_slave_axil_master.v</code>	I2C 从设备到 AXI-Lite 主接口的桥接模块
<code>i2c_slave_wbm.v</code>	I2C 从设备到 Wishbone 主接口的桥接模块
<code>i2c_init.v</code>	基于 ROM 指令表的 I2C 初始化状态机
<code>i2c_single_reg.v</code>	单字节寄存器型 I2C 从设备示例
<code>axis_fifo.v</code>	AXI Stream FIFO，用于缓冲和接口解耦

附录 B 后续实验建议

为了使论文成果更加完整，建议后续补充以下实验材料：

1. 搭建 MyHDL 与 Icarus Verilog 环境，运行项目测试脚本。
2. 保存 `test_i2c_master.py`、`test_i2c_slave.py`、`test_i2c_init.py` 等测试日志。
3. 对关键事务生成波形截图，展示起始条件、地址阶段、数据阶段、ACK 和停止条件。
4. 选择目标 FPGA 器件进行综合，记录 LUT、触发器、块 RAM 等资源占用。
5. 添加一个真实 I2C 外设或仿真存储器的演示工程，验证控制器实际读写流程。
6. 根据实验结果补充论文第六章，使“验证结果”从结构分析提升为可复现实验结论。